

AITG: AUTOMATING INTENT-ORIENTED TASK GENERATION FOR MOBILE GUI-AGENT

Chuning Zhang¹, Yuchen Huang², Mei Tu^{1,*}

¹ Samsung Research China - Beijing, China

² National University of Singapore, Singapore

cning.zhang@partner.samsung.com, yuchen.huang@u.nus.edu, mei.tu@samsung.com

ABSTRACT

The advancement of large language models has significantly propelled research in mobile graphical user interface (GUI) agents, concurrently amplifying the demand for high-quality benchmark and data. However, existing benchmark data collection methodologies remain heavily reliant on manual effort, especially for designing high-quality queries for mobile GUI-agent. Furthermore, most benchmarks and training data for mobile GUI agent focus on English environments and lack localized application. To address these limitations, we propose an automated pipeline for generating intent-oriented tasks by selecting suitable task execution trajectories with Focus Score. This pipeline includes three phases, namely *Seed Path* selection, *Focus Score* extension, and *Intent-oriented Query* generation. It achieves automated data generation while ensuring the authenticity of the collected trajectories. Utilizing the generated dataset, we have evaluated and fine-tuned existing GUI agents. The experimental results demonstrate the practical utility of our automatically generated data for enhancing the performance of real-world models.¹

Index Terms— GUI-Agent, Seed-path, Focus-score, Localization, less-human-effort

1. INTRODUCTION

In recent years, Large Language Models (LLMs) and the vision capabilities of Vision-Language Models (VLMs) [1] [2] like GPT-4V [3] have significantly advanced the development of autonomous AI agents. A key research frontier is the creation of Graphical User Interface (GUI) [4] [5] [6] agents that can understand screen content and operate applications like humans, a capability that promises to automate complex digital workflows. The field has seen rapid progress, evolving from early models operating in controlled environments to sophisticated VLM-based architectures. These range from specialized agents designed for specific tasks (e.g., MobileVLM [7], MobileFlow [8]) to versatile, general-purpose agents (e.g., AppAgent [9], SeeClick [10]) and foundational models aiming at broader applicability (e.g., GUI-Owl [11], UI-Tars [12, 13]).

Corresponding author

¹Our code and dataset are available at: <https://github.com/conservative-force/AITG.git>

However, training or evaluating these high-performance GUI agents faces a critical bottleneck: the need for high-quality labeled data. Historically, creating such datasets was a manual effort, relying on human annotators for benchmarks like AITW [14] and AndroidControl [15]. This process is notoriously expensive, time-consuming, and difficult to scale. While recent automated crawling methods (e.g., the crawler from AITW, MobileViews [16]) have increased data volume, they often lack the explicit planning required for high-quality, task-oriented data. Recently, some works are exploring on-line training mobile GUI agent without human supervision. ZeroGUI [17] uses example-guided prompting to generate tasks given a certain App. The tasks lack generalization due to the limited understandability of LLM to a certain App. MobileGUI-RL [18] uses self-exploration to capture trajectories and GPT-4o to reverse-engineer task descriptions. It is conducted on Android emulator with native application. Although they employ heuristics such as preferring unexplored UI elements, it is hard to ensure the trajectories coherent with clear intent. Moreover, the instruction is generated from each step, which tends to be machine-like instruction instead of user-intent instruction.

To address these challenges, we propose AITG, an automated data generation framework to select goal-focused trajectories and intent-oriented instruction. The main contributions of this paper are summarized as follows:

- We propose a novel automated trajectory collection method. With a Seed Path selection strategy to initialize exploration and a Focus Score filtering mechanism, it effectively addresses the challenge of ensuring task orientation in automatically collected trajectories on real mobile devices.
- We design a two-stage instruction generation pipeline driven by two distinct LLM-based agents (a Generator and a Rectifier). This structured approach significantly improves the quality of converting complex operation sequences into high-level task descriptions.
- We build an end-to-end automated GUI data generation framework, **AITG**, which achieves the fully automated production of high-quality mobile agent data.

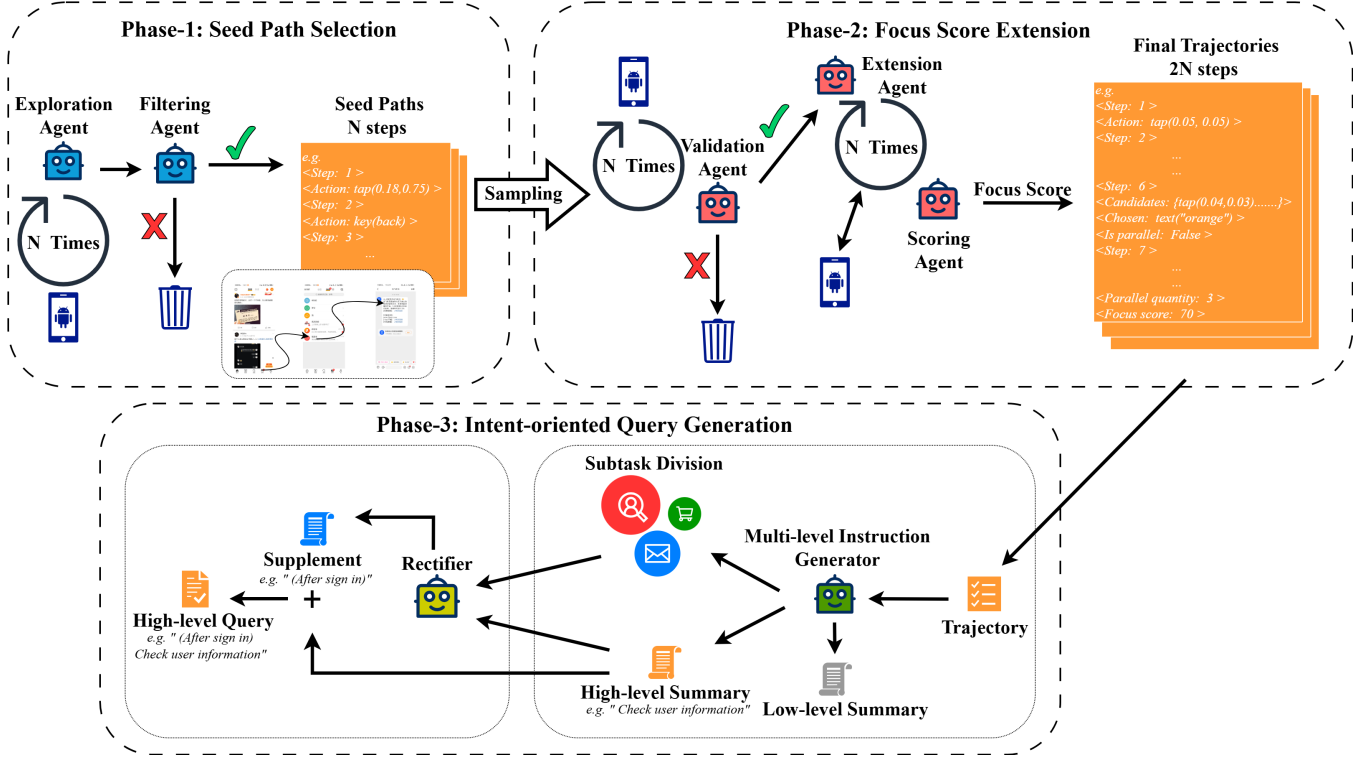


Fig. 1: The overview of our method. Our three-phase framework first uses agents to find promising "seed paths" that end in functionally rich UIs. These paths are then extended into longer, goal-oriented trajectories validated by a scoring agent. Finally, high-scoring trajectories are converted into concise, reproducible instructions by abstracting user intent while re-integrating critical operational details.

2. METHOD

Our methodology is a unified three-phase framework designed for the autonomous generation of high-quality mobile interaction data. Orchestrated by VLM and LLM-based agents, the framework sequentially executes: **1) Seed Path Selection**, **2) Focus Score Extension**, and **3) Intent-oriented Query Generation**. The entire process is based on the foundational capability of VLMs to interpret a screen state S_t and propose a feasible action space A_t in a continuous perception-action loop.

To navigate the vast state space of modern applications efficiently, our strategy addresses the fundamental challenge of balancing exploration breadth with trajectory depth. We introduce two guiding concepts to manage this trade-off: *Seed Paths* and the *Focus Score*. A *Seed Path* is a short trajectory containing UI states with various functions, used for further diversified ground exploration. The *Focus Score* is a metric that quantifies the coherence of trajectory for the task, ensuring that extended interactions remain goal-oriented.

The following sections will elaborate on the implementation of each of these three phases in detail.

2.1. Phase 1: Seed Path Selection

The objective of Phase 1 is to efficiently explore the application's functional space and generate a high-quality pool of seed paths. This phase is orchestrated by two distinct agents: an **exploration agent** and a **filtering agent**.

The process unfolds over K rounds of exploration. In each round, the **exploration agent** executes a stochastic strategy: starting from the initial state, it randomly selects and performs a sequence of N actions identified by the VLM, generating a trajectory segment of length N .

Following this, the **filtering agent** applies a heuristic filter to ensure these trajectories conclude in functionally rich states and avoid dead-ends. Specifically, upon completion of these N steps, the filtering agent evaluates the resulting state S_{N+1} . A trajectory is validated as a seed path only if the cardinality of the feasible action space in this state $|A_{N+1}|$ exceeds the predefined threshold X .

2.2. Phase 2: Focus Score Extension

For the seed paths obtained from Phase 1, we adopt a random sampling approach, selecting one path at a time for further processing.

Phase 2 is to extend the high-potential seed paths from Phase 1 into longer, task-focused trajectories. This phase is orchestrated by a multi-agent workflow comprising a **Validation Agent**, an **Extension Agent**, and a **Scoring Agent**. The process unfolds as follows: first, the **Validation Agent** ensures the selected seed path is reproducible by executing its action sequence. If successful, the **Extension Agent** continues the interaction for an additional N steps from terminal state of the seed. Finally, the **Scoring Agent** evaluates the quality of the resulting combined trajectory using our proposed *Focus Score*.

The *Focus Score*, denoted $\mathcal{F}(\tau)$, is designed to quantitatively distinguish goal-oriented trajectories from those with aimless or redundant interactions. The score is calculated by classifying actions into two categories based on their impact on the GUI:

1. **State-Changing Operations:** Actions that fundamentally advance a task or navigate the user to a new functional state. This category includes steps such as navigating to a new screen or submitting a form.
2. **Parallel Operations:** User actions that occur without a state transition and has no contribution to the task's forward momentum. Examples include aimless scrolling without subsequent interaction, or toggling a purely cosmetic setting.

During the extension, the VLM is used to classifying each executed action into one of these types.

Formally, for a trajectory τ of length L containing N_p parallel operations, the Focus Score is defined as the normalized frequency of state-changing operations:

$$\mathcal{F}(\tau) = \frac{L - N_p}{L} = 1 - \frac{N_p}{L}$$

A score closer to 1 signifies a highly coherent, task-relevant trajectory. These high-scoring trajectories are prioritized and passed to the final phase for intent-oriented query generation.

2.3. Phase 3: Intent-oriented Query generation

Following the collection and filtering of high-focus trajectories, the final phase centers on synthesizing the raw interaction data into structured, executable natural language instructions. While raw trajectories contain a complete record of operational details, they are too low-level to directly train agents capable of understanding high-level tasks. This phase therefore addresses a core challenge: how to balance the abstraction of a user's high-level intent with the preservation of critical operational details necessary for task reproducibility.

To achieve this balance between conciseness and completeness, we designed a cooperative pipeline driven by two Large Language Model (LLM)-based agents: the **Multi-level Instruction Generator** and the **Rectifier**.

2.3.1. The Multi-level Instruction Generator: Abstraction from Trajectory to Intent

This agent hierarchically abstracts raw trajectories in a bottom-up manner. Its workflow is divided into three steps:

1. **Low-level Summary:** First, it translates and assembles the low-level operations recorded in the raw trajectory (e.g., coordinate-based clicks, component IDs, text inputs) into a sequence of human-readable, step-by-step low-level instructions.
2. **Action Segmentation:** Next, the agent analyzes this stream of low-level instructions to identify and group consecutive actions that serve a common sub-goal into multiple **Action Clusters**. Each cluster represents a logical sub-task within the overall workflow (e.g., "log in to account," "select item specifications," or "fill out address").
3. **High-level Summary:** Finally, based on the identified action clusters and the trajectory as a whole, the agent infers the user's intent and generates a concise, **High-level Instruction**.

2.3.2. The Rectifier: Mitigating Information Loss and Enhancing Detail

While high-level instructions provided by the generator are concise, they often omit details essential for reproducibility. To mitigate the loss of information, we introduce the **Rectifier**. It takes the high-level instruction and its corresponding action clusters as input, compares them to identify missing details (e.g., menu navigation, a required checkbox), and integrates this necessary information from the original low-level actions back into the final instruction, often as a parenthetical note.

This "generate-and-rectify" mechanism yields high-quality instructions that achieve a robust balance between high-level abstraction and the low-level completeness required for successful execution and subsequent agent training.

3. EXPERIMENTS

3.1. Settings

In this paper, we use Doubao-seed-1.6² as multiple VLM agents. For hyper-parameters N, X, K , we test different values and set them empirically as $N = 3, X = 6, K = 30$. We collect data from four localized applications and test mainstream systems (UI-Tars, GUI-Owl) on our data. The core metrics measured were the agent's ability to correctly predict the required action for each execution step and the accuracy of the parameters supplied for those actions.

²<https://www.volcengine.com/>

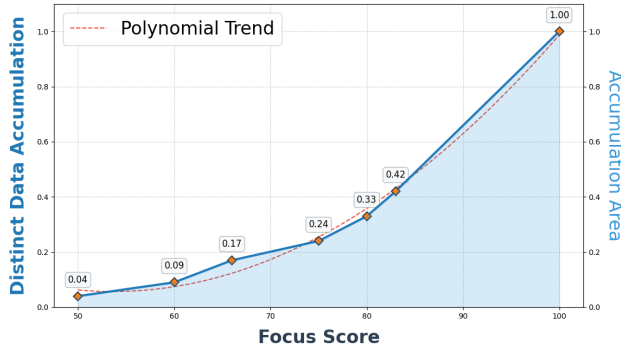


Fig. 2: Distinct data accumulation along with different focus score

3.2. Data Statistics

The distinct UI elements accumulation is illustrated in Fig. 2 along with different *FocusScore*. When *FocusScore* is less than 80, the distinct UI elements accumulate slowly, which indicates that trajectories with $\mathcal{F}(\tau) < 80$ are likely to fall in similar UI elements, while $\mathcal{F}(\tau) > 80$ dominates distinct UI elements, which implies that trajectory with higher *Focus Score* is better, which is a good indicator to find unexplored UI elements.

Table 1: Dataset statistics by application

Category	Type	Applications			
		B.	Dy.	Tb.	QQ m.
Total	Trajectory	4k	3k	4k	3k
	Action	25k	17k	20k	20k
Test	Trajectory	61	263	144	131
	Action	317	1k	734	697
FT data with <i>FocusScore</i> > 0.8	Trajectory	240	152	193	1k
	Action	1.2k	800	1k	5.7k

We list the trajectories and actions of localized applications in Table 1, including Bilibili(B.), Douyin(Dy.), Taobao(Tb.), QQ music(QQ m). We sampling around 600 trajectories from all the total data as testing data. Based on the Fig. 2, data with *FocusScore* > 80 dominates distinct UI elements. Thus we filtering data with *FocusScore* under 0.8 as the finetuning data.

3.3. Result

Experimental results for the evaluated GUI-agent models are presented in Table 2. Performance is measured across Type Exact Match (Type EM), Tap Accuracy (Tap Acc), and Step Success Rate (StepSR). The UI-Tars-7B model demonstrated

the strongest overall performance, achieving scores of 0.77 in Type EM, 0.88 in Tap Acc, and 0.77 in StepSR. In contrast, the GUI-Owl-7B model attained lower scores across all evaluated metrics, with 0.69 in Type EM, 0.78 in Tap Acc, and 0.68 in StepSR. These results indicate a consistent performance advantage for UI-Tars-7B over GUI-Owl-7B.

Table 2: Benchmark result

	Type EM	Tap Acc	StepSR
UI-Tars-7B	0.77	0.88	0.77
GUI-Owl-7B	0.69	0.78	0.68
GUI-Owl-32B	0.71	0.82	0.72
GUI-Owl-7B w/ FT	0.96	0.99	0.93

Table 3: App performance benchmark

	Bilibili	Douyin	Taobao	QQmusic
UI-Tars-7B	0.85	0.78	0.70	0.77
GUI-Owl-7B	0.66	0.71	0.63	0.67
GUI-Owl-32B	0.72	0.74	0.66	0.70

Table 2 also presents the performance comparison of the GUI-Owl-7B model before and after training. After training (“w train”), the model demonstrated substantial improvements across all metrics, attaining significantly higher scores of 0.96 in Type EM, 0.99 in Tap Acc, and 0.93 in StepSR. These results represent absolute performance gains of 0.27 (+39%), 0.21 (+27%), and 0.25 (+37%) respectively, indicating that the training set enhanced the model’s interaction capabilities.

To further analyze the baseline performance on specific applications, we provide a detailed breakdown in Table 3. These results show the performance of untrained models across four major platforms. UI-Tars-7B consistently outperforms the other baseline models, especially on the Bilibili (0.85) and Taobao (0.70) applications. The performance of all models varies across different apps, indicating that model capabilities are application-dependent, which motivates the need for specialized fine-tuning.

4. CONCLUSION

In this paper, we introduce an automated pipeline that collects mobile GUI-agent trajectories and generates corresponding intent-oriented instructions. This ensures data authenticity while enabling scalable automation. Experiments demonstrate its practical value for developing and enhancing real-world GUI agents. This work provides a key solution to data scarcity and a valuable resource for advancing Android GUI agents.

5. REFERENCES

- [1] Kenton Lee, Mandar Joshi, Iulia Raluca Turc, Hexiang Hu, Fangyu Liu, Julian Martin Eisenschlos, Urvashi Khandelwal, Peter Shaw, Ming-Wei Chang, and Kristina Toutanova, “Pix2struct: Screenshot parsing as pretraining for visual language understanding,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 18893–18912.
- [2] Gilles Baechler, Srinivas Sunkara, Maria Wang, Fedir Zubach, Hassan Mansoor, Vincent Etter, Victor Cărbune, Jason Lin, Jindong Chen, and Abhan-shu Sharma, “Screenai: A vision-language model for ui and infographics understanding,” *arXiv preprint arXiv:2402.04615*, 2024.
- [3] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al., “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [4] Yang Li, Jiacong He, Xin Zhou, Yuan Zhang, and Jason Baldridge, “Mapping natural language instructions to mobile ui action sequences,” *arXiv preprint arXiv:2005.03776*, 2020.
- [5] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou, “Showui: One vision-language-action model for gui visual agent,” in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 19498–19508.
- [6] Peter Shaw, Mandar Joshi, James Cohan, Jonathan Berant, Panupong Pasupat, Hexiang Hu, Urvashi Khandelwal, Kenton Lee, and Kristina N Toutanova, “From pixels to ui actions: Learning to follow instructions via graphical user interfaces,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 34354–34370, 2023.
- [7] Xiangxiang Chu, Limeng Qiao, Xinyang Lin, Shuang Xu, Yang Yang, Yiming Hu, Fei Wei, Xinyu Zhang, Bo Zhang, Xiaolin Wei, and Chunhua Shen, “Mobilevlm : A fast, strong and open vision language assistant for mobile devices,” 2023.
- [8] Kostas Pentikousis, Yan Wang, and Weihua Hu, “Mobileflow: Toward software-defined mobile networks,” *IEEE Communications Magazine*, vol. 51, no. 7, pp. 44–53, 2013.
- [9] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu, “Appagent: Multimodal agents as smartphone users,” in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 2025, pp. 1–20.
- [10] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu, “Seeclick: Harnessing gui grounding for advanced visual gui agents,” *arXiv preprint arXiv:2401.10935*, 2024.
- [11] Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, et al., “Mobile-agent-v3: Foundational agents for gui automation,” *arXiv preprint arXiv:2508.15144*, 2025.
- [12] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al., “Ui-tars: Pioneering automated gui interaction with native agents,” *arXiv preprint arXiv:2501.12326*, 2025.
- [13] Haoming Wang, Haoyang Zou, Huatong Song, Jiazhan Feng, Junjie Fang, Junting Lu, Longxiang Liu, Qinyu Luo, Shihao Liang, Shijue Huang, et al., “Ui-tars-2 technical report: Advancing gui agent with multi-turn reinforcement learning,” *arXiv preprint arXiv:2509.02544*, 2025.
- [14] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap, “Androidinthewild: A large-scale dataset for android device control,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 59708–59728, 2023.
- [15] Wei Li, William E Bishop, Alice Li, Christopher Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva, “On the effects of data scale on ui control agents,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 92130–92154, 2024.
- [16] Longxi Gao, Li Zhang, Shihe Wang, Shangguang Wang, Yuanchun Li, and Mengwei Xu, “Mobileviews: A large-scale mobile gui dataset,” *arXiv preprint arXiv:2409.14337*, 2024.
- [17] Chenyu Yang, Shiqian Su, Shi Liu, Xuan Dong, Yue Yu, Weijie Su, Xuehui Wang, Zhaoyang Liu, Jinguo Zhu, Hao Li, et al., “Zerogui: Automating online gui learning at zero human cost,” *arXiv preprint arXiv:2505.23762*, 2025.
- [18] Yucheng Shi, Wenhao Yu, Zaitang Li, Yonglin Wang, Hongming Zhang, Ninghao Liu, Haitao Mi, and Dong Yu, “Mobilegui-rl: Advancing mobile gui agent through reinforcement learning in online environment,” *arXiv preprint arXiv:2507.05720*, 2025.